Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Разработка и оценка интеллектуальной системы для ускорения выявления и устранения программных ошибок / Development and Evaluation of an Intelligent System for Faster Software Bug Identification and Resolution**

Работу выполнил / Thesis is executed by	**Андуков Алмаз Алексеевич / Andukov Almaz Alexeevich**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Махмуд Надери / Mahmoud Naderi**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **7**

1.1 Background and Motivation 7

1.2 Problem Statement 8

1.3 Aim and Objectives 9

1.4 Scope and Expected Contribution 10

1.5 Thesis Structure 10

2 Literature Review **12**

2.1 The Role of Bug Reports in Software Maintenance 12

2.2 Completeness and Quality of Bug Reports 13

2.3 Structured Forms and Guided Information Collection 15

2.4 Natural Language Processing and Large Language Models . . . 17

2.5 LLM-Based Approaches to Bug Report Improvement 18

2.6 Evaluation of Bug Reporting Approaches 20

2.7 Research Gap and Motivation 22

Bibliography cited **24**

Abstract

Software bugs are often reported through technical support channels, but the quality and completeness of incoming reports can significantly affect how quickly developers understand and resolve problems. Reports may lack important information, such as steps to reproduce an issue, expected behavior, error messages, or relevant contextual details. As a result, developers may need to spend additional time clarifying the problem before they can begin investigating it.

The goal of this thesis is to develop and evaluate an intelligent technical support form designed to improve the process of identifying and resolving software bugs. The proposed solution will guide users through submitting structured bug reports and use a Large Language Model (LLM) to assist developers in interpreting the submitted information. The LLM will access the service's knowledge base and documentation through the Model Context Protocol (MCP), allowing it to provide recommendations grounded in the available technical information.

The effectiveness of the proposed solution will be evaluated by comparing it with a conventional technical support workflow. The evaluation will focus on the time required to understand and diagnose a reported problem, the completeness of the reports submitted, the accuracy of the suggested diagnosis, and the time needed to identify an appropriate solution. The expected result is an assessment of whether a structured support form combined with documentation-based AI assistance can improve the efficiency of software bug investigation and resolution.

Chapter 1

Introduction

1.1 Background and Motivation

Software bugs are an inevitable part of software development and maintenance. When users encounter an unexpected behavior, they typically report the problem to a technical support team or create a bug report in an issue-tracking system. The quality of such reports directly affects how quickly developers can understand the problem, identify its potential cause, and implement a solution.

In practice, bug reports submitted by users often lack important technical details. Users may describe symptoms in general terms without specifying the steps required to reproduce the issue, the expected and actual behavior, the software environment, or relevant error messages. As a result, developers and support engineers may need to request additional information, clarify the original report, and reproduce the problem before they can begin investigating it. These interactions increase the time required to resolve issues and create additional workload for both users and technical teams.

Traditional support forms collect information through predefined fields. Although such forms help standardize incoming reports, they may not adequately address the differences between individual issues. A form that is too general

may fail to collect essential information, while an overly complex form may discourage users from completing it or require them to provide irrelevant details. Therefore, there is a need for a more flexible approach to collecting information about software bugs.

Intelligent assistance can potentially improve this process by analyzing a user's initial description, identifying missing information, and guiding the user toward providing a more complete and structured report. Such assistance may help bridge the gap between how users describe problems and the technical information developers need to investigate them. However, the practical value of this approach should be evaluated in comparison with conventional support forms.

1.2 Problem Statement

The main problem addressed in this thesis is the insufficient quality and completeness of information provided in software bug reports. When a report does not contain enough information to reproduce or understand an issue, technical teams must spend additional time communicating with the reporter and collecting the missing details.

Conventional support forms provide a standardized way to submit issues, but they generally rely on users to understand what information is relevant and how to describe a technical problem. This assumption does not always hold, particularly when users have limited technical knowledge or cannot identify the source of the observed behavior.

An intelligent technical support form could address this limitation by adapting the information-gathering process to the user's description. Instead of relying exclusively on a fixed set of fields, it could provide contextual prompts, request missing details, and help organize the submitted information into a structured bug report.

Nevertheless, introducing intelligent assistance does not automatically guarantee better results. Its effectiveness depends on whether it improves the completeness and usefulness of reports without making the submission process unnecessarily complicated. Therefore, the problem is not only to develop such a form but also to evaluate whether it offers practical advantages over a conventional approach.

1.3 Aim and Objectives

The aim of this thesis is to develop and evaluate an intelligent technical support form designed to improve the quality of software bug reports and reduce the effort required to identify and investigate reported issues.

To achieve this aim, the following objectives are defined:

1. Analyze the common problems associated with collecting information about software bugs through conventional support forms.
2. Identify the essential information required to understand, reproduce, and investigate software issues.
3. Design and implement an intelligent technical support form that guides users through the process of describing a software problem.
4. Develop a mechanism for identifying missing or unclear information and helping users provide more complete bug reports.
5. Evaluate the developed solution by comparing its results with those obtained using a conventional support form.
6. Analyze the results to determine the practical benefits and limitations of the proposed approach.

1.4 Scope and Expected Contribution

This thesis focuses on the process of collecting and structuring information about software bugs. The proposed solution is intended to assist users in preparing reports that are sufficiently detailed for further investigation by developers or technical support specialists.

The scope of the work includes the design and implementation of a user-facing support form, the processing of submitted descriptions, and the presentation of structured information about reported issues. Intelligent assistance is considered a tool for improving information collection rather than a replacement for developers, automated debugging systems, or software testing tools.

The evaluation focuses on the quality and completeness of submitted reports and the effort required to obtain the information needed for issue investigation. The work does not aim to develop a system capable of automatically fixing software defects or guaranteeing the correct identification of their root causes.

The expected contribution of this thesis is a working prototype of an intelligent technical support form and an evaluation of its practical usefulness compared with a conventional form. The results should provide insight into how contextual guidance and automated processing can improve the initial stage of software bug reporting.

1.5 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 presents the background and related work concerning software bug reporting, technical support processes, and intelligent approaches to information collection. Chapter 3 describes the requirements, architecture, and design of the proposed solution. Chapter 4 presents the implementation of the intelligent technical support form and its main components. Chapter 5 describes the evaluation methodology,

presents the results, and discusses the effectiveness and limitations of the proposed approach. Finally, Chapter 6 summarizes the work, draws conclusions, and identifies possible directions for future development.

Chapter 2

Literature Review

2.1 The Role of Bug Reports in Software Maintenance

Bug reports are an important source of information during software maintenance. They describe unexpected software behavior, allow developers to investigate reported problems, and support the process of reproducing and fixing defects. The usefulness of a bug report, however, depends not only on the existence of a reported problem but also on the quality and completeness of the information provided.

Bettenburg et al. investigated the characteristics of effective bug reports by surveying developers and users of several open-source projects. Their findings revealed a mismatch between the information developers consider useful and the information users are able to provide. In particular, developers value steps to reproduce a problem, stack traces, and test cases, while these elements can be difficult for users to supply [1]. This mismatch indicates that improving bug reporting requires more than providing a text field in which users can describe an issue. Reporting tools should also help users understand what information is needed.

The importance of individual report elements was examined further by Soltani et al. [2]. Their study combined interviews with developers and a survey of software practitioners to investigate the perceived importance of different bug report elements. The results indicated that crash descriptions, reproduction steps or test cases, and stack traces are particularly valuable to developers. However, the usefulness of individual elements depends on the nature of the reported problem and the context in which it occurs. Consequently, a reporting system should collect relevant information without requiring users to provide every possible technical detail for every issue.

The relationship between report content and the software maintenance process is also discussed by Wang et al. [3]. Their analysis of bug reports from web applications identified information that was frequently absent from initial reports but appeared later during discussions between users and developers. The findings demonstrate that a bug report can evolve as additional information is requested and supplied. This observation motivates the development of reporting interfaces that identify missing information earlier in the reporting process.

Together, these studies establish that the quality of a bug report is an important factor in making reported problems understandable and reproducible. They also suggest that a reporting interface should help users communicate relevant information rather than simply accept an initial description.

2.2 Completeness and Quality of Bug Reports

A central challenge in bug reporting is determining whether a description contains enough information to understand the reported behavior. Although users may describe a problem in natural language, their descriptions do not necessarily explain what happened, what they expected to happen, or how the problem can be reproduced.

Chaparro et al. [4] investigated the presence of essential information in bug descriptions, focusing on observed behavior, expected behavior, and steps to reproduce. Their analysis of nearly 3,000 bug reports found that observed behavior was explicitly described in most reports, whereas expected behavior and reproduction steps were substantially less frequent. The authors also developed an approach for detecting whether expected behavior and reproduction steps were present in a report. Their work demonstrates that missing information can be treated as a detectable property of a bug description rather than only as a subjective assessment of its quality.

The quality of reproduction steps is another important aspect of bug report completeness. Chaparro et al. proposed EULER, an approach that automatically identifies and assesses the quality of reproduction steps and provides feedback to reporters [5]. Their findings demonstrate that the presence of reproduction steps and their actual quality are different properties. Consequently, evaluating a reporting form should consider not only whether the required fields contain information, but also whether the information is sufficiently clear and useful for understanding the reported problem.

The elements required for a useful report can be organized into several categories:

- **Observed behavior:** what the user experienced when the problem occurred.
- **Expected behavior:** what the user expected the software to do.
- **Steps to reproduce:** the actions or sequence of events that lead to the problem.
- **Technical context:** relevant information such as the operating system, application version, browser, or execution environment.

- **Supporting evidence:** error messages, stack traces, screenshots, logs, or other diagnostic information when available.

These categories provide a useful foundation for assessing report completeness. Nevertheless, completeness should not be interpreted as the presence of every possible field. Some issues do not produce error messages, and some users may not know the technical details of their environment. The information required to investigate a problem depends on its type and context.

The study by Wang et al. [3] reinforces this distinction by showing that information needs can extend beyond the initial textual description. Additional context about how an application was used and diagnostic suggestions may emerge during subsequent communication. Therefore, the quality of a report should be assessed according to the relevance and usefulness of its content, rather than solely by its length or the number of completed fields.

For the present research, report quality is considered primarily in terms of completeness, clarity, and the availability of information that helps another person understand the reported issue. These properties provide a basis for comparing an intelligent reporting form with a conventional form.

2.3 Structured Forms and Guided Information Collection

A conventional bug reporting form usually consists of predefined fields, such as a title, description, reproduction steps, environment, and attachments. Such forms provide a consistent structure and make reports easier to organize. However, they rely on users recognizing which information is important and knowing how to describe it.

Zimmermann et al. investigated the characteristics of effective bug reports and developed CUEZILLA, a prototype designed to assess newly submitted re-

ports and recommend additional information that could improve them [6]. Their work demonstrates that automated feedback can be integrated into the reporting process. However, identifying missing report elements does not necessarily explain to users what information is required or help them formulate a useful response. An interactive reporting form could address this limitation by providing targeted questions instead of only indicating that information is missing.

A structured form can reduce ambiguity by separating different types of information into explicit fields. For example, asking users to describe the actual behavior separately from the expected behavior may help distinguish the observed problem from the user's interpretation of it. Similarly, requesting reproduction steps can encourage users to describe the actions that led to the issue rather than providing only a general statement that something does not work.

However, a fixed set of mandatory fields may introduce unnecessary effort. Users can be required to answer questions that are irrelevant to their issue, while complex or unfamiliar fields may discourage them from completing the form. A form that simply increases the number of required fields does not necessarily improve the quality of the resulting report.

An alternative is guided information collection, in which the system responds to the information already supplied. If a user reports that a file cannot be uploaded, for example, the form could ask what happened after the upload was attempted, whether an error message appeared, and whether the problem occurs with other files. The exact questions should depend on the initial description and on information that remains unclear.

This approach changes the reporting process from a static questionnaire into an iterative interaction. The system first receives an initial description, identifies potentially missing information, and asks targeted follow-up questions. The final report can then be organized into a consistent structure for further investigation.

2.4 Natural Language Processing and Large Language Models

Natural language processing (NLP) techniques have been investigated as a means of extracting useful information from unstructured bug descriptions. Such techniques can identify textual patterns, classify report elements, and determine whether specific information is present. This is relevant because users often describe problems in their own words rather than following a predefined template.

Chaparro et al. [4] demonstrated that automated techniques can detect the presence or absence of important elements, including expected behavior and reproduction steps. Their work illustrates how information extracted from a report can be used to provide feedback at reporting time. However, recognizing whether a report contains a particular element is different from understanding its full meaning or deciding which follow-up question would be most useful.

Large language models (LLMs) offer additional possibilities for interpreting natural-language descriptions and generating context-sensitive responses. Unlike approaches based only on predefined patterns, LLMs can process varied phrasing and produce questions or summaries adapted to a user's description. These capabilities make them potentially suitable for interactive reporting systems.

McGowan et al. [7] explored the use of large language models to assist end users in bug reporting. Their proposed approach uses information about an application's interface and functionality to help generate more complete bug reports, including reproduction steps. The study highlights the possibility of combining information about the software with an LLM to improve the content of user-submitted reports. At the same time, it illustrates that the quality of generated information can depend on the context available to the model.

Hou et al. [8] conducted a systematic literature review of LLM applications in software engineering. Their work provides a broader overview of how LLMs are

being applied to software engineering tasks and the limitations that accompany these applications. In the context of bug reporting, this broader perspective supports treating LLMs as tools that can assist a particular task rather than assuming that generated output is automatically correct.

One important limitation is that a language model may produce plausible but unsupported details. For example, if a user reports that a page fails to load but does not provide an error message, a model should not invent a specific error code or claim that a particular sequence of steps has been confirmed. An intelligent form should distinguish between information explicitly provided by the user, information inferred from the description, and information that remains unknown.

For this reason, the role of an LLM in the proposed system is limited to assisting with information collection and organization. It may identify ambiguity, suggest relevant questions, and help structure the final report, but it should not be treated as an authoritative source of facts about the reported problem.

2.5 LLM-Based Approaches to Bug Report Improvement

Recent research has investigated the use of LLMs not only to interpret bug reports but also to assess and improve their quality.

Bo et al. [9] introduced ChatBR, an approach that combines a classifier with ChatGPT to assess and improve bug reports. The system detects the presence of observed behavior, expected behavior, and steps to reproduce, then uses ChatGPT to generate missing information. The generated content is subsequently assessed as part of the improvement process. The authors report improvements in missing-information detection and evaluate the semantic similarity of generated information across several projects.

ChatBR is particularly relevant because it combines automated quality assessment with language-model-based improvement. Nevertheless, the objective of generating missing report elements differs from the objective of interactively collecting information from the person who experienced the problem. When information is absent, generating a plausible description is not necessarily equivalent to obtaining a reliable account of the actual behavior. A user-facing system should therefore prioritize clarification and confirmation when the available evidence is insufficient.

Mukherjee and Rahman proposed BugMentor, a system that combines structured information retrieval with neural text generation to help answer follow-up questions associated with software bug reports [10]. Their work highlights the importance of additional information exchanged between reporters and developers. While BugMentor focuses on generating answers to questions about existing reports, the present thesis investigates whether relevant information can be collected earlier through an intelligent reporting form.

Acharya and Ginde [11] studied LLM-based generation of structured bug reports from informal descriptions. Their experiments compared instruction-tuned models and ChatGPT using several quality and similarity metrics. The results indicate that LLMs can transform unstructured descriptions into more structured reports, although the performance varies across models, report elements, and evaluation settings. This suggests that model selection and evaluation criteria can substantially influence the effectiveness of an LLM-based reporting approach.

Chen et al. [12] investigated the capability of LLMs to decompose complex bug reports into smaller, more manageable units. Their findings indicate that model performance depends on the prompting strategy and that the models can still make errors when interpreting the structure and content of bug reports. Although report decomposition differs from collecting missing information, the study provides additional evidence that LLM-based assistance requires explicit

instructions and careful evaluation.

These studies suggest that LLMs can support multiple stages of the bug reporting process, including quality assessment, information generation, restructuring, and decomposition. However, they address different tasks and use different evaluation methods. Their results should therefore not be interpreted as direct evidence that every LLM-based reporting interface will improve report quality or reduce user effort.

The present research focuses on a narrower application: using an intelligent form to identify missing or unclear information and guide users through the process of completing a report. The purpose is not to generate a complete account of a problem without user input, but to help users provide information that they can confirm as accurate.

2.6 Evaluation of Bug Reporting Approaches

An intelligent reporting form should be evaluated according to measurable outcomes rather than the perceived sophistication of its interface or language model. Previous studies demonstrate several ways of assessing bug report quality, including detecting missing elements, comparing generated content with reference information, and examining the importance of report elements to developers [2], [4], [9], [11].

For the present thesis, the evaluation should distinguish between the quality of the resulting report and the effort required to produce it. A system may generate more complete reports while also requiring users to answer too many questions. Conversely, a short interaction may be convenient but fail to collect information needed to understand the issue.

Several dimensions are therefore relevant to the evaluation:

- **Completeness:** whether the final report contains relevant information about

observed behavior, expected behavior, reproduction steps, and other applicable details.

- **Clarity:** whether the report communicates the problem in a way that another person can understand without relying on unsupported assumptions.
- **Information accuracy:** whether the final report preserves the user's statements and avoids introducing unconfirmed details.
- **User effort:** the time required to complete the reporting process and, where appropriate, the number of additional questions that the user must answer.
- **Practical usability:** whether users can complete the form without unnecessary confusion or difficulty.

A comparison with a conventional form can help determine whether intelligent guidance provides a measurable benefit. To make such a comparison meaningful, both approaches should be evaluated using comparable reporting tasks and the same criteria for assessing report quality. If participants complete both versions, the study should also consider the order of tasks and the possibility that experience with the first form influences performance with the second.

It is important to distinguish improvements in report quality from improvements in actual software debugging or resolution time. A more complete report may help developers investigate an issue, but the effect on resolution time also depends on the problem's complexity, the developer's experience, the available diagnostic tools, and the software environment. Unless the study directly measures these outcomes, conclusions should be limited to the quality of the reports and the effort required to prepare them.

2.7 Research Gap and Motivation

The reviewed literature establishes that incomplete bug reports remain a practical problem in software maintenance. Developers need specific information to understand and reproduce reported issues, yet users may omit reproduction steps, expected behavior, technical context, or supporting evidence [1], [2], [3], [4]. Research has explored automated detection of missing information, structured report generation, and the use of LLMs to assess or improve report quality [7], [9], [11].

However, these approaches address different aspects of the reporting process. Some focus on detecting missing report elements, while others generate or restructure report content. Research on LLMs also demonstrates the need to consider reliability, prompt sensitivity, and the possibility of unsupported output [8], [12]. Consequently, the ability to generate a more complete-looking report should not automatically be interpreted as evidence that the report accurately reflects the user's experience or that the reporting process is more efficient.

The motivation for the present thesis is to investigate whether an interactive, intelligent reporting form can help users provide more complete and understandable bug reports than a conventional form. The proposed approach focuses on identifying missing or ambiguous information, asking relevant follow-up questions, and organizing the user's answers into a structured report. The system is intended to support the user rather than independently determine the cause of a software defect.

The contribution of this thesis is the design, implementation, and evaluation of a prototype that applies this approach to technical support reporting. Its effectiveness will be assessed by comparing reports produced through the intelligent form with those produced through a conventional form. The comparison will focus on report completeness, clarity, accuracy, and the effort required to complete the reporting process.

In this way, the thesis builds on prior research into bug report quality and LLM-assisted report improvement while concentrating on the interaction between the reporting interface and the user. The central question is whether context-sensitive guidance can improve the information supplied by users without making the reporting process unnecessarily complicated.

Bibliography cited

[1] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, doi: 10.1145/1453101.1453146. [Online]. Available: https://doi.org/10.1145/1453101.1453146, ACM, 2008, pp. 308–318.

[2] M. Soltani, F. Hermans, and T. Bäck, "The significance of bug report elements," *Empirical Software Engineering*, vol. 25, no. 6, pp. 5255–5294, 2020, doi: 10.1007/s10664-020-09882-z. [Online]. Available: https://doi.org/10.1007/s10664-020-09882-z.

[3] D. Wang, M. Galster, and M. Morales-Trujillo, "Information needs in bug reports for web applications," *Journal of Systems and Software*, vol. 219, p. 112 230, 2025, doi: 10.1016/j.jss.2024.112230. [Online]. Available: https://doi.org/10.1016/j.jss.2024.112230.

[4] O. Chaparro et al., "Detecting missing information in bug descriptions," in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*, doi: 10.1145/3106237.3106285. [Online]. Available: https://doi.org/10.1145/3106237.3106285, ACM, 2017, pp. 396–407.

[5] O. Chaparro et al., "Assessing the quality of the steps to reproduce in bug reports," in *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, doi: 10.1145/3338906.3338947, ACM, 2019, pp. 86–96.

[6] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, "What makes a good bug report?" *IEEE Transactions on Software Engineering*, vol. 36, no. 5, pp. 618–643, 2010, doi: 10.1109/TSE.2010.63.

[7] A. McGowan, J. Mahmud, and K. Moran, "Adapting large language models to aid end-users in bug reporting," in *Journal of Student-Scientists' Research*, doi: 10.13021/jssr2023.3932. [Online]. Available: https://doi.org/10.13021/jssr2023.3932, vol. 5, 2023.

[8] X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3695988. [Online]. Available: https://doi.org/10.1145/3695988.

[9] L. Bo, W. Ji, X. Sun, T. Zhang, X. Wu, and Y. Wei, "Chatbr: Automated assessment and improvement of bug report quality using chatgpt," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, doi: 10.1145/3691620.3695518. [Online]. Available: https://doi.org/10.1145/3691620.3695518, ACM, 2024, pp. 1472–1483.

[10] U. Mukherjee and M. M. Rahman, "Bugmentor: Generating answers to follow-up questions from software bug reports using structured information retrieval and neural text generation," *Journal of Systems and Software*, vol. 231, p. 112 636, 2026, doi: 10.1016/j.jss.2025.112636.

[11] J. Acharya and G. Ginde, "Can we enhance bug report quality using llms? an empirical study of llm-based bug report generation," *arXiv preprint arXiv:2504.18804*, 2025, [Online]. Available: https://arxiv.org/abs/2504.18804.

[12] Z. Chen, V. Nava-Camal, A. Suleiman, Y. Tang, D. Hou, and W. Shang, "An empirical study on the capability of llms in decomposing bug reports,"

arXiv preprint arXiv:2504.20911, 2025, [Online]. Available: https://arxiv.org/abs/2504.20911.